PyPIGuard: A Lightweight, Explainable Machine Learning Tool for Pre-Install Detection of Malicious PyPI Packages

Rahat Ahmed

Department of Computer Science and Engineering, BRAC University, Dhaka, Bangladesh

Abstract

PyPIGuard is an open-source tool that classifies PyPI packages as malicious or benign using static, AST, and bytecode-level features, without execution or a large language model. Deployed as a live web application with real-time, file-upload, and notebook scanning, it is trained on 9,723 packages, training under 3 seconds versus hours for behavioral deep-learning baselines. A direct check found the name-disjoint headline evaluation (0.998 ROC-AUC) inflated by template-level duplication a name-based split does not catch; a cluster-disjoint re-evaluation, grouping the training population by feature-vector similarity rather than package names, gives a more defensible 0.983 ROC-AUC, 0.940 recall, reported in full rather than the inflated figure alone. This work includes independent dataset verification, a head-to-head comparison against an existing scanner, and two deployment findings corrected for the specific instances found (v1.1, v1.2), including a generalization test showing the underlying pattern is not fully resolved. An installable CLI and CI/CD example demonstrate concrete reuse; full detail is in the code repository (C2).

Keywords: malware detection, software supply chain security, machine learning, static code analysis, PyPI, explainable AI

Required Metadata

Current code version

Main text

1. Motivation and significance
   Open-source package registries such as PyPI are foundational to modern software development, but are increasingly targeted by attackers

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.2
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/rahat239/PyPIguard-Research`
C3	Legal Code License	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python 3.12 (development version), scikit-learn, Flask, HTML/CSS/JavaScript
C6	Compilation requirements, operating environments & dependencies	Python $\geq$3.10; `pip install -r requirements.txt` (pandas, numpy, scikit-learn, joblib, requests, flask, shap) for the web app, or `pip install .` for the standalone `pypiguard` library/CLI (numpy, scikit-learn, joblib, requests only); no OS-specific dependencies
C7	If available Link to developer documentation/manual	`README.md` in the code repository (C2)
C8	Support email for questions	rahatahmed537@gmail.com

Table 1: Code metadata (mandatory)

who publish malicious packages, often via typosquatting popular libraries such as `beautifulsoup4`, `matplotlib`, and `requests`. In the `event-stream` incident (2018) [1], an attacker was granted maintainer access to a popular package by offering to help maintain it, and the compromised package reached over 1,600 downstream projects before discovery.

Existing approaches sit at two extremes, leaving a gap PyPIGuard targets. Lightweight metadata-only classifiers are fast but easily gamed. Heavyweight behavioral models (e.g., Cerebro [2], requiring 14–25 hours of training and an 81.5% FPR on PyPI in its own evaluation) are too slow for real-time scanning. Vu et al.'s benchmark [3] (evaluating PyPI's own malware checks, Bandit4Mal, and OSSGadget's OSS Detect Backdoor) found 15–97% false-positive rates across these scanners, and reports administrators consider rates above $\sim$0.01% operationally unacceptable given daily release volume.

In practical terms: a developer supplies a package name, archive, or notebook; PyPIGuard analyzes it without executing it and returns a CLEARED/FLAGGED verdict, a confidence score, and the patterns that drove the decision.

The gap PyPIGuard addresses: a static-analysis classifier fast enough for real-time use, explainable per-prediction, and validated against adversarial evasion, temporal drift, and cross-ecosystem transfer, none jointly reported for the tools in [3]. It is built on the dataset released with [4] and PyPI's official index.

2. Software description

   2.1 **Software architecture.** Figure 1 shows the end-to-end pipeline. A submitted package is first checked against a **verified-lookup table** of 5,900 unique packages (9,723 counts labeled samples; the lookup table covers unique names, extending to held-out packages; Section 4 confirms evaluation never uses this shortcut); if found, an instant dataset-backed verdict is returned, marked "verified" not model-predicted. Otherwise: (1) **feature extraction** computes 35 static features via regex, `ast`, and `compile()`-based bytecode inspection, none requiring execution; (2) **classification**, a single Random Forest, outputs a prediction and confidence; (3) **delivery** (Flask) formats the verdict with SHAP explanation. An **out-of-distribution safety gate** flags unreliable verdicts when package size falls far outside the training range; trigger rate is unquantified, and whether it fired for the `requests`/`typing-extensions` cases is not confirmed. As of v1.2, stages (1)–(2) are a standalone `pypiguard` package consumed by both the Flask app and the CLI.

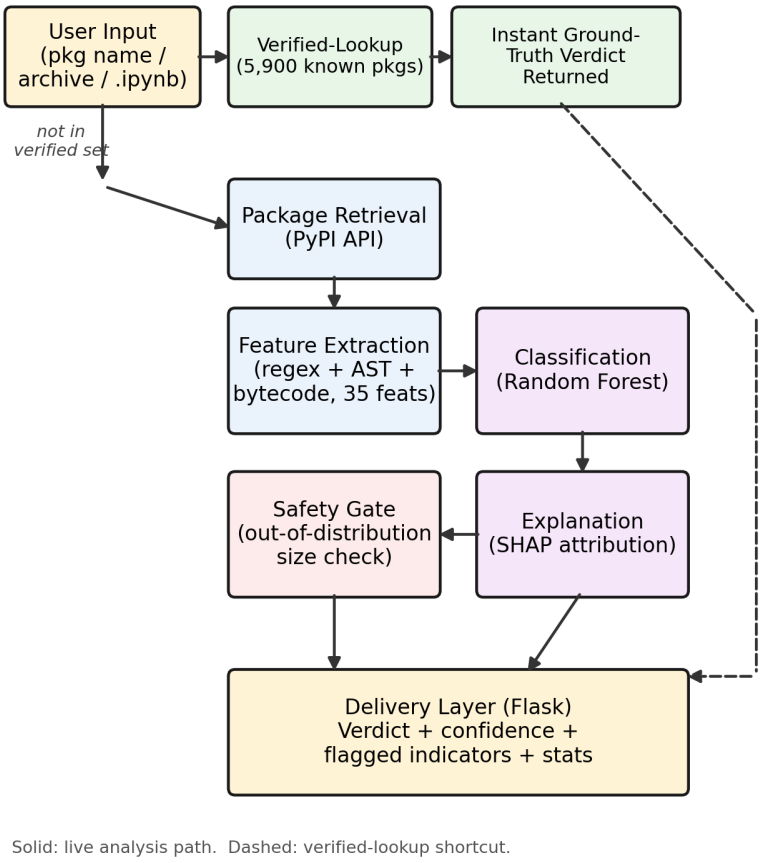


Figure 1: PyPIGuard architecture. Solid arrows: live analysis path. Dashed arrow: verified-lookup shortcut.

2.2 **Software functionalities.** Three input modes share one verdict format. *Package-name scanning* downloads and analyzes a package live via PyPI's API. *Archive-file scanning* accepts a local archive (`.tar.gz`, `.zip`, or `.whl`) directly, useful for historical malware PyPI has removed; extraction is hardened against path-traversal ("zip-slip") attacks, validated against a crafted adversarial archive. *Notebook scanning* parses every `.ipynb` code cell, extracts `import`/`pip install` references, excludes Python's ∼300 standard-library modules, resolves import-to-distribution mismatches (e.g., `cv2`→`opencv-python`), and reports how many referenced packages are known or suspected malware before execution. Every verdict includes a CLEARED/FLAGGED label, confidence, flagged patterns (e.g., `eval()` usage, `setup.py cmdclass` hooks, high-entropy blobs), and a bytecode-level detector for dynamically-invoked dangerous functions independent of the string-construction technique used to hide the target. Limitations: no dynamic/execution-based analysis, and reduced reliability for packages far larger than the training data (flagged via the safety gate above).

2.3 **Sample code snippets.** The detection core is invoked identically from the CLI, the Python API, and the web app. A representative end-to-end CLI scan:

```
$ pip install pypiguard
$ pypiguard scan requests
Package: requests
Verdict: CLEARED (confidence 54.7%)
Flagged patterns: none above threshold
```

Internally, `scan` resolves the package name, downloads the source distribution, and calls the same feature-extraction/classification pipeline the web app uses:

```
import ast_features, regex_features, joblib
feats = {
    **regex_features.extract(source_dir),
    **ast_features.extract_ast_features(source_dir)
}
model = joblib.load("final_integrated_model.joblib")
proba = model.predict_proba([feats])[0]
verdict = "malicious" if proba[1] > 0.5 else "benign"
explanation = shap_explainer(feats)
```

The web application wraps this same pipeline behind the three input modes; full implementation, including the file- and requirements-scanning commands, is in the code repository (C2).

3. Illustrative examples
   A user submits a package name (e.g., `requests`) or uploads a local archive. Figure 2(a) shows the resulting interface for `requests`: a CLEARED verdict at 54.7% model confidence and the flagged patterns behind it, the same case discussed in Section 4. Figure 2(b) shows the notebook-upload interface reaching a verified-lookup result for a known-malicious package (`selfcraftint`): a database-verified match, not a model confidence score, returned via the lookup table (Section 2) rather than live import-parsing; the interface's stated limitations are also shown.
4. Impact
   Evaluation methodology. Results below reflect the final model after both corrections (Fix v1.1, v1.2). The model was trained on 9,723 labeled packages (7,960 malicious, 1,763 benign; the entire available malicious archive from [4], plus a sampled benign set) and evaluated on a 400-package held-out set with zero name overlap with training: 0.998 ROC-AUC, 0.999 PR-AUC, 0.957 precision, 0.995 recall (n=396; 4 excluded for archive-retrieval failures unrelated to model performance – Figure 3a). This ∼82%-malicious composition inverts PyPI's deployment base rate; `class_weight="balanced"` is used during training, and a 1:1 balanced-subsample check gives 0.992 ROC-AUC versus 0.998 on the full set. Evaluation sets are artificially balanced (200/200), so reported precision/FPR are not the deployment-scale false-alarm rate: at 1 malicious per 1,000 scanned, 4.5% FPR implies ∼45 false alarms per detection. All predictions came directly from the classification pipeline, bypassing the verified-lookup shortcut. The 200 held-out malicious names were cross-referenced against OpenSSF [5]; 172/200 (86%) are corroborated.
   A confirmed name-disjoint-but-feature-duplicate leakage problem. Name-disjointness does not rule out near-duplicate code, and a direct check confirms this. Malicious PyPI campaigns routinely copy-paste one payload across many typosquat names (e.g., 40 `requests` variants, 20 `colorama` variants in this dataset); family-aware `GroupKFold` grouping, already used for internal cross-validation, was never applied when building or verifying the external held-out set – only a plain name-set intersection was checked there, which removed 168 exact-name duplicates but cannot detect a same-template package under a different name (confirmed within the held-out set itself: 13 of 400 feature vectors are exact-duplicate templates under different names). Checking the 200 held-out malicious packages against an independent 1,500-package training-adjacent pool: **76 (38.0%) are exact feature-vector du-**

(a) Package scan result (`requests`)

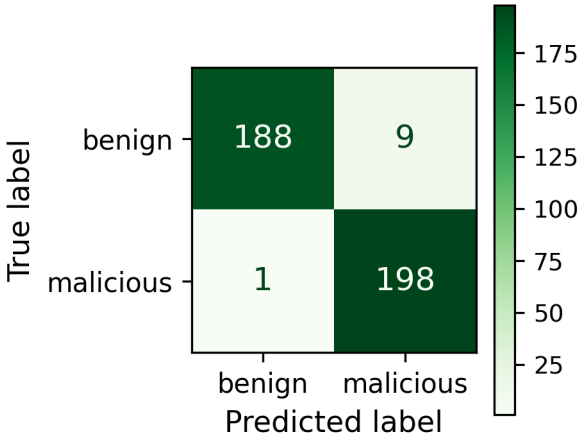
(b) Notebook upload & verified-lookup result

Figure 2: PyPIGuard web interface: (a) live scan of `requests`, showing the CLEARED verdict, model confidence score, and flagged patterns discussed in Section 4; (b) notebook-scanning upload widget reaching a database-verified (not model-predicted) result for a known-malicious training-set package.

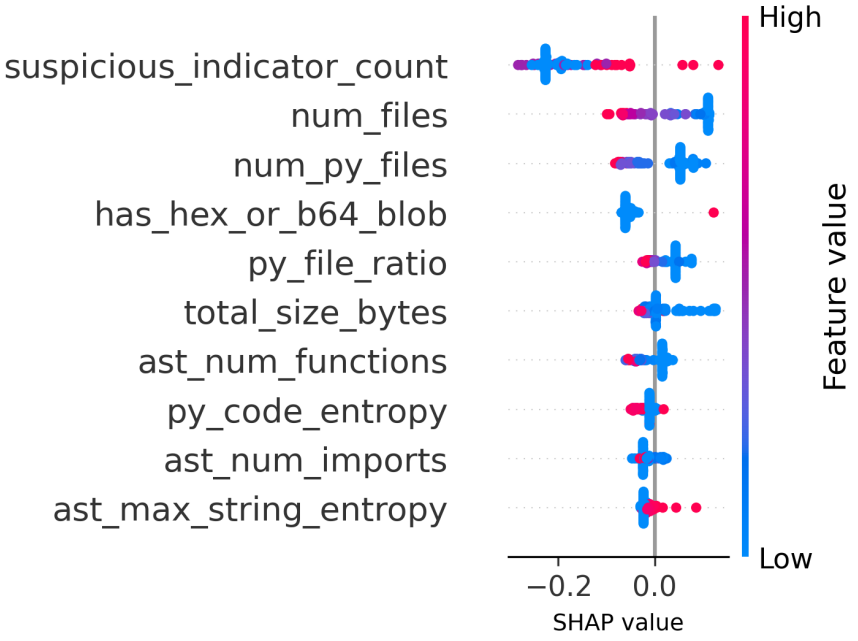
plicates and 186 (93.0%) are near-duplicates (distance <0.5), median nearest-neighbor distance 0.000, checked against roughly one-fifth of the full training set, so the true rate is plausibly higher. A substantial share of the reported 0.995 recall therefore reflects recognizing a memorized template under a new alias, not generalizing to novel malware. A cluster-disjoint re-evaluation quantifies this: the full 7,960-package malicious population, grouped by feature-vector similarity, collapses to 808 distinct clusters, 89.8% sharing a cluster with another package. Splitting whole clusters 85/15 and retraining from scratch: **0.983 ROC-AUC, 0.978 precision, 0.940 recall, 0.959 F1, 3.8% FPR** on a cluster-disjoint 744-package test set. ROC-AUC barely moves (0.998→0.983); recall drops meaningfully (0.995→0.940), consistent with a model that still discriminates well but misses more novel malware once templates cannot leak across the split. This cluster-disjoint result supersedes the name-disjoint headline as the defensible generalization estimate; only 138 of 9,723 packages had a pre-existing family label, so clusters were built directly from feature similarity. Full methodology is in the code repository (C2).

Robustness. *Temporal*: training on 7,290 samples dated before 2024-03-26 (6,265 malicious, 1,025 benign), testing on the 2,431 after, yields 0.984 ROC-AUC, 0.994 PR-AUC. *Adversarial evasion*: disguising 100 malicious packages reduced evasion from 9% (regex-only) to 2% (AST-augmented). *Cross-ecosystem*: 600 npm packages (300 malicious, DataDog's dataset; 300 benign, live registry), mapped to the same 21-column schema, applied with zero retraining: 0.815 ROC-AUC (vs. 0.999 in-ecosystem; recall 0.54), so deployment there needs retraining. *Secondary-dataset validation*: a disjoint 400-package set yields 0.959 precision, 0.930 recall, 0.944 F1, 4.0% FPR under v1.0 (re-used below for v1.1). This addresses held-out label verification only; the ∼7,960 training malicious labels remain single-source, unchecked. Full methodology is in the code repository (C2).

Baseline comparison. Cerebro's [2] 81.5% PyPI FPR is self-reported on different data, not head-to-head; a re-run of either baseline's code is impossible since neither released it (MalGuard's [7] unavailability confirmed in third-party surveys; Cerebro's is consistent with the citing literature but not independently confirmed to the same standard). Two mitigations are used instead. First, GuardDog [6] (DataDog's PyPI/npm scanner, v3.1.0) was run against the identical 400-package held-out set, recovering removed-malware archives from `pypi_malregistry` [4] since 199/200 malicious packages were already gone live, the same archive PyPIGuard trained on. Table 2 reports two rule-inclusion read-



(a) Confusion matrix



(b) SHAP summary

Figure 3: (a) Confusion matrix, primary held-out set (n=396 of 400; 4 excluded for archive-retrieval failures unrelated to model performance): 188/197 benign cleared (9 FP), 198/199 malicious flagged (1 FN). (b) SHAP global feature importance: position shows impact on output, color shows feature value.

ings; under both, PyPIGuard's FPR (4.6%) is substantially lower than GuardDog's (67.0%/23.5%), with closer recall (0.995 vs. 0.910/0.875); at n=396–400 with no confidence intervals, small deltas are indicative, not confirmed. Second, since Cerebro's core idea (behavioral-sequence modeling) is partially reproducible without its code, a simplified sequence-based proxy was built and evaluated on a separate matched subset: it modestly outperforms PyPIGuard there (0.983 vs. 0.967 ROC-AUC, Future Work below), the closest this manuscript comes to directly testing Cerebro's approach rather than citing self-reported numbers. Full methodology is in the code repository (C2).

Metric	**PyPIGuard**	**GuardDog (any rule)**	**GuardDog (threat only)**
Precision	0.957	0.576	0.788
Recall	0.995	0.910	0.875
F1	0.975	0.705	0.829
False positive rate	0.046	0.670	0.235

Table 2: Head-to-head comparison on the primary 400-package held-out set (200 malicious, 200 benign). GuardDog scanned all 400; PyPIGuard's n=396 excludes 4 packages that failed archive retrieval during re-evaluation (Figure 3a caption) – a <1% difference in evaluated set, noted for precision rather than presented as strictly identical.

Resource-performance comparison against Cerebro and MalGuard. Training the full 9,723-sample model takes 2.3 s on a commodity CPU core, versus Cerebro's self-reported 14–25 hours [2] and 2,439 s (Cerebro) / 30,741.67 s (EA4MP) as independently re-run by the MalGuard authors [7]. Per-package feature extraction is 0.9 ms (median) versus 12.489 s (Cerebro) and 6.28 s (EA4MP) [7], a three-to-four order-of-magnitude gap. MalGuard's own GuardDog evaluation (95.6% precision, 82.6% recall on different data [7]) is broadly consistent with the measured GuardDog range in Table 2. Full methodology in the code repository (C2).

Explainability. Figure 3b shows SHAP importance for the top 10 of 35 features. `suspicious_indicator_count` (triggered regex patterns) is the most influential feature, followed by `num_files`, `num_py_files`, and `has_hex_or_b64_blob`: more triggered patterns push toward malicious, consistent with thin typosquats; larger file counts push toward benign, consistent with legitimate packages carrying more supporting files than minimal payloads. As with any tree-ensemble SHAP attribution, correlated features can share credit unstably; rankings are directional, not exact.

Latency (corrected measurement). Initial benchmarking reported 0.18 ms median model-inference time. Re-benchmarking the actually-deployed classifier found this does not hold: single-request inference measures 6.8–7.4 ms, roughly 40× higher, because a 300-tree Random Forest's per-call overhead does not amortize at batch size 1. Disclosed rather than silently corrected: the original figure was measured against an earlier Logistic Regression candidate, before the switch to Random Forest, and never re-benchmarked. Feature extraction remains fast (0.9 ms median); training remains 2.3 s. Full root-cause methodology is in the code repository (C2); a ∼20s `requests` scan observed during testing is download/cold-start time, not inference.

Confidence calibration (deployment-testing finding). Confidence is the raw class-probability for the *predicted* class, not always P(malicious); no post-hoc calibration is applied. `requests` was classified benign at 54.3% during testing; a re-scan returned 54.7%, both near the boundary. The stability question that matters is across *versions*: testing five historical `requests` releases found 3 of 5 **misclassified as malicious**. Initial investigation attributed this to vendored dependencies; deeper root-causing found this holds for only 1 of 3, the others caused by `setup.py` boilerplate and `tests/` content, non-runtime code treated identically to shipped code. A feature-extraction filter was rejected: no net FP reduction, new false negatives on malware cloning the test suite. Full breakdown is in the code repository (C2).

Fix (v1.1, hard-negative retraining). `requests` and its common dependencies were entirely absent from training despite being this manuscript's own illustrative example. Adding 7 hard-negative examples and retraining (still 2.3 s) fixes all 5 `requests` versions. Cost: on the secondary set, recall drops 93.0%→91.0% (0.963 precision, 3.5% FPR; 5 newly missed – `nhmpy`, `xoloaoqcjnreyc`, `xolopfydnuxyfh`, `wayspiritmcp-ppa`, `claude-lite`, all near-boundary). A sample-weight sweep confirmed this trade-off is intrinsic. After v1.2, the result shifts to 0.953 precision, 0.910 recall, 0.931 F1, 4.5% FPR – rising despite more targeted negatives, since v1.2's fix broadly adjusts sensitivity to bundled content rather than narrowly targeting the two motivating cases. Full derivation, including a patched path-traversal vulnerability, is in the code repository (C2).

Fix (v1.2, found via real-world reuse testing). Rather than asserting reuse potential, it was tested: the new CLI was run against `black`'s actual PyPI-declared dependencies. Of 8, 7 cleared correctly; `typing-extensions` was flagged malicious at 68.0% confidence, root-caused to the same pattern as `requests` (its sdist bundles CPython's

own 9,768-line test suite). The identical fix corrects it: `typing-extensions` now classifies correctly (80.7%, benign), `requests` remains correct, and combined precision moves 0.967→0.955, F1 0.960→0.954. Both fixes were validated on the families they targeted. Testing a third, previously-unused foundational package with heavy bundled content – Django 6.0.7 (23 MB `tests/` directory) – found it **misclassified as malicious at 52.7% confidence**: the underlying pattern is not resolved in general. Full transcript in the code repository (C2).

Practical impact and reuse. Intended users are developers, students, and CI/CD pre-install gates. The 4.5% combined FP / 4.8% FN rates sit far above the ∼0.01% bar cited in Section 1 for a fully autonomous blocker; PyPIGuard is positioned instead as a decision-support triage tool surfacing a confidence score and explanation for review. It is deployed as a public, live tool (see "Current executable software version" below), testable against historical malware the live registry no longer serves. v1.2 adds a pip-installable `pypiguard` library and CLI (`scan`, `scan-file`, `scan-requirements --fail-on-malicious`) and a GitHub Actions workflow; exercising it against a real project's dependencies is what surfaced the `typing-extensions` finding above. Full CLI/CI documentation is in the code repository (C2).

Future work includes extending static learning to other ecosystems beyond the partial PyPI-to-npm transfer above, and building further on the sequence-proxy signal noted above (Baseline comparison) with a full sequence-based model rather than the current simplified reproduction.

5. Conclusions

PyPIGuard is an open-source, deployed, explainable tool for pre-install detection of malicious PyPI packages, validated through temporal, cross-ecosystem, adversarial-evasion, and secondary-dataset testing not typically reported together at this scope. A direct check found the name-disjoint headline evaluation inflated by template-level duplication a name-based split does not catch; a cluster-disjoint re-evaluation, grouping the full malicious training population by feature-vector similarity rather than package names, gives the more defensible 0.983 ROC-AUC, 0.940 recall reported in Section 4, the single most consequential correction in this work. It reports a substantially lower false-positive rate than a head-to-head comparison against an existing scanner [6], while disclosing further boundaries: partial cross-ecosystem transfer, reduced-but-not-eliminated adversarial evasion, a corrected ∼40× latency discrepancy, and two deployment findings (`requests`, `typing-extensions`) fixed for those specific cases (v1.1, v1.2), with a generalization test on a third package (Django) finding the underlying

pattern persists. The detection core is a pip-installable library and CLI with a working CI/CD example; exercising it against an unrelated project's dependencies is what surfaced the v1.2 finding in the first place.

CRediT authorship contribution statement

Rahat Ahmed: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used ChatGPT (OpenAI) to assist with drafting and restructuring portions of the manuscript, correcting grammatical errors, and refining writing style. After using this tool, the author reviewed, edited, and verified all content for technical accuracy and takes full responsibility for the content of the published article.

Acknowledgements

The author thanks the maintainers of the `pypi_malregistry` dataset [4], which made this work possible. This research received no external funding.

References

[1] M. Ohm, H. Plate, A. Sykosch, M. Meier, Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks, in: DIMVA 2020, LNCS vol. 12223, Springer, 2020, pp. 23-43.

[2] J. Zhang, K. Huang, B. Chen, C. Wang, Z. Tian, X. Peng, Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI Using a Single Model of Malicious Behavior Sequence, ACM Transactions on Software Engineering and Methodology, 2025.

[3] D.-L. Vu, Z. Newman, J.S. Meyers, A Benchmark Comparison of Python Malware Detection Approaches, arXiv:2209.13288, 2022.

[4] W. Guo, Z. Xu, C. Liu, C. Huang, Y. Fang, Y. Liu, An Empirical Study of Malicious Code In PyPI Ecosystem, in: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2023, pp. 166-77. Dataset mirror used: `https://github.com/lxyeternal/pypi_malregistry`.

[5] Open Source Security Foundation, Malicious Packages Database, GitHub repository, https://github.com/ossf/malicious-packages, 2024.

[6] DataDog, GuardDog: Identify malicious PyPI and npm packages, GitHub repository, v3.1.0, https://github.com/DataDog/guarddog, 2024.

[7] X. Gao, X. Sun, S. Cao, K. Huang, D. Wu, X. Liu, X. Lin, Y. Xiang, MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem, in: 34th USENIX Security Symposium, USENIX Association, 2025, pp. 4741-4758.

Current executable software version

Live demonstration available at: https://rahat239.github.io/Malware-classifier/, deployed from a separate repository (https://github.com/rahat239/Malware-classifier) that packages the same detection core distributed in C2 as a Flask web application; the research code repository (C2) is authoritative for training, evaluation, and the installable CLI, while this repository is the live deployment target.